

```
    }  
  }  
  
  // Carve out the paths in the maze data structure:  
  let hasVisited = [[1, 1]]; // Start by visiting the top-left corner.  
  visit(1, 1);  
  
  // Display the final resulting maze data structure:  
  printMaze(maze);  
</script>
```

When you run this program, it produces a large amount of text that will fill the terminal window or browser with each step of the maze's construction. You'll have to scroll back up to the top to view the entire output.

The maze data structure begins as a completely filled-in 2D space. The recursive backtracker algorithm is given a starting point in this maze and then visits a previously unvisited neighboring space, “carving out” any hallway space in the process. Then it recursively calls itself on a neighboring space it hasn't visited before. If all the neighboring spaces have already been visited, the algorithm is at a dead end and backtracks to an earlier visited space to visit its unvisited neighbors. The program ends when the algorithm backtracks to its starting location.

You can see this algorithm in action by running the maze-generator program. As the maze is carved out, it displays the current x, y coordinates by using the @ character. The process looks like [Figure 11-1](#). Notice that the fifth image in the top-right corner has backtracked to an earlier space after reaching a dead end to explore a new neighboring direction from that space.